Benchmarking anomaly detection techniques in the fraud detection domain

Data Science Applications, 2023/24

Code: https://github.com/leobt23/Applications_DSE

1st Leonardo Brito
Engineering and Data Science
Instituto Superior Técnico
Lisbon, Portugal
leonardo.amado.brito@tecnico.ulisboa.pt

Company Supervisor: Jacopo Bono
Data Scientist
Feedzai
Porto, Portugal
jacopo.bono@feedzai.com

***Abstract*—This study aimed to benchmark fraud detection using various machine learning models, including both supervised and unsupervised approaches. We utilized techniques like NearMiss and SMOTE for class balancing and assessed model performance using ROC AUC, F1 Score, and Accuracy. The Support Vector Classifier (SVC) emerged as the most effective model, while OneClassSVM and Isolation Forest had limitations.**

Future research can explore deep learning and ensemble methods and focus on real-time detection systems.

***Index Terms*—fraud, detection, supervised, unsupervised, performance**

I. Introduction

In the realm of digital transactions, the significance of detecting fraud at its earliest is paramount. Supervised learning models are often the preferred choice, renowned for their accuracy, but they hinge on the availability of labeled data. This dependency can introduce operational delays, leaving a window where transactions might be exposed to unchecked fraudulent activities.

To address this gap, there's a growing interest in leveraging unsupervised learning. Without the need for labeled data, unsupervised anomaly detection techniques offer a promising avenue, especially when one considers that fraud typically surfaces as statistical anomalies amidst legitimate transactions.

Drawing from open-source datasets, this report sets out to critically evaluate a range of unsupervised anomaly detection methods in the context of fraud detection. Our approach is methodical: starting with a detailed data exploration, we then benchmark against a supervised baseline, and finally, analyze the performance of both classical and deep learning unsupervised models. The overarching aim is to ascertain the potential of unsupervised techniques in offering interim protection against fraud during periods when supervised models are not yet feasible.

II. Exploratory Data Analysis

A. Overview of data

This dataset represents credit card transactions made by European cardholders in September 2013 over two days.

Out of the 284,807 transactions recorded, 492 were fraudulent, making up a mere 0.172% of the total transactions. Given the disparity in numbers, the dataset is notably imbalanced, which I am going to talk more about later.

The dataset consists mainly of numerical variables derived from a Principal Component Analysis (PCA) transformation. To maintain confidentiality, the original features and further details about the data are not disclosed. The variables V1, V2, through V28 are the principal components derived from the PCA. The only exceptions that haven't undergone PCA are 'Time' and 'Amount'. The 'Time' feature indicates the seconds that have passed between each transaction and the first one in the dataset. On the other hand, 'Amount' denotes the transaction value, which can be useful for context-sensitive learning approaches. The 'Class' feature is the target variable, where a value of 1 indicates a fraudulent transaction, and 0 signifies a legitimate one.

There are no null or missing values in the dataset, but was 1854 observations duplicated, which were removed because they don't add any value to the analysis.

B. Analysis of Transaction Amounts for Fraudulent and Valid Transactions

The table I presents a summary of transaction amounts for both fraudulent and valid categories. A notable observation is the disparity in the count of transactions: while there are only 473 fraudulent transactions, there are a staggering 284,253 valid ones. This vast difference highlights the class imbalance inherent in the dataset.

When examining the transaction amounts, fraudulent transactions have a mean value of approximately $122.21, which is slightly higher than the valid transactions' mean of $88.29. Despite this, the maximum fraudulent transaction ($2125.87) is significantly lower than the highest valid transaction, which reaches a substantial $25,691.16.

TABLE I
SUMMARY STATISTICS FOR FRAUD AND VALID TRANSACTIONS

Statistic	Fraud	Valid
Count	473	284253
Mean	122.21	88.29
Standard Deviation	256.68	250.11
Minimum	0.0	0.0
25% Quantile	1.0	5.65
50% Quantile (Median)	9.25	22.0
75% Quantile	105.89	77.05
Maximum	2125.87	25691.16

The spread of transaction amounts, as represented by the standard deviation, is roughly similar for both classes, hovering around the $250 mark. The median value for valid transactions ($22.00) is more than double that of fraudulent transactions ($9.25), indicating that half of the fraudulent transactions are below this amount.

While fraudulent transactions are comparatively rarer, their amounts can vary significantly, sometimes even surpassing the typical amounts seen in valid transactions. It underscores the importance of leveraging advanced techniques to detect such anomalies amidst the vast sea of valid transactions.

C. Imbalanced Dataset

An imbalanced dataset is a common challenge where one class significantly outnumbers the other. This disproportion can adversely affect the model's performance and the interpretability of statistical metrics. Our dataset, as we can see in figure 1, with one class constituting 99.83% of observations, exemplifies this issue.

The primary concerns with such an imbalance include:

- **Performance Deception:** Conventional metrics like accuracy may be deceptive. A simplistic model that only predicts the predominant class will erroneously appear highly accurate, masking its ineffectiveness in identifying the minority class.
- **Model Bias:** There is a tendency for machine learning models to favor the majority class.
- **Decreased Predictive Power:** The minority class, which is usually of greater interest, is often underrepresented in predictions. This leads to a higher incidence of false negatives.

Furthermore, the imbalance can distort the interpretation of correlation metrics.

- Minor patterns in the minority class can produce strong correlations due to the imbalance, potentially leading to overemphasized or spurious relationships in the model.
- The predominance of the majority class can skew correlation measures, possibly obscuring significant relationships in the minority class.
- The scarcity of data points for the minority class increases the risk of missing actual relationships (false negatives), limiting the reliability of these measures.

Later in this report, we will explore strategies to address these challenges and ensure a more balanced and effective analysis.

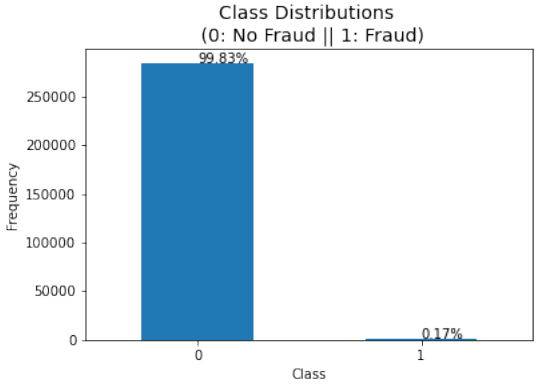


Fig. 1. Class balance of the dataset.

D. Distributions

Due to the PCA transformation, the features are not directly interpretable. However, we can still examine the distributions of the features to gain some insights. The distributions of the features for both fraudulent and valid transactions are shown in Figure 39, in Annex A. The goal was to understand the distribution of the features for both classes. Several features exhibit distinct distribution patterns when comparing the two class categories: V4 and V11 show clear segregation between Class 0 and Class 1, suggesting high discriminatory power. Features V12, V14, and V18 display partial separation, indicating moderate distinction between classes. V1, V2, V3, and V10 each present a somewhat unique distribution, whereas V25, V26, and V28 appear to have overlapping distributions for both classes. Generally speaking, except for the 'Time' and 'Amount' features, the distribution of features for genuine transactions (Class = 0) tends to be centralized around 0, often with an extended tail at one end. Conversely, distributions for fraudulent transactions (Class = 1) are typically asymmetrical, reflecting a deviation from the norm. Although we can not understand what each variable PCA transformed means, this can provide good insights for feature engineering and model prediction.

E. Variance

Principal Component Analysis (PCA) is a statistical technique primarily used for dimensionality reduction in datasets, particularly effective in scenarios like anonymizing client data. Its application yields a transformed data set where each component captures a different aspect of variance in the data. The key elements of variance analysis in PCA-transformed data include:

1) **Variance Explained:** Understanding the variance explained by each principal component is crucial. This is typically visualized using a **Scree Plot**, which displays the fraction of total variance accounted for by each component. Such a plot is instrumental in deciding the number of components to retain for analysis.
2) **Eigenvalues and Variance:** The eigenvalue of each principal component indicates the amount of variance

it captures. In PCA, these components are arranged in order of descending eigenvalues, signifying that the first few components encapsulate the most significant variance in the data.

3) **Orthogonality and Uniqueness:** Each principal component is orthogonal to the others, ensuring that they are uncorrelated. This characteristic means that every subsequent component captures the highest variance possible that is not represented by the preceding components.
4) **Maximizing Variance Direction:** The first principal component ($PC1$) captures the maximum variance, with each following component ($PC2$, $PC3$, etc.) being orthogonal to the previous ones and representing the next highest variance direction. This order of components is pivotal in understanding the underlying structure of the data.

PCA's role in dimensionality reduction and data anonymization is not just about reducing the number of variables but also about uncovering the most informative variance structure within the data. This approach offers a nuanced view of data, crucial for sensitive applications like client data anonymization.

The image below, figure 2, corroborates the fact the PCA was already applied. There is no correlation between the PCA-transformed variables because each variable is a principal component.

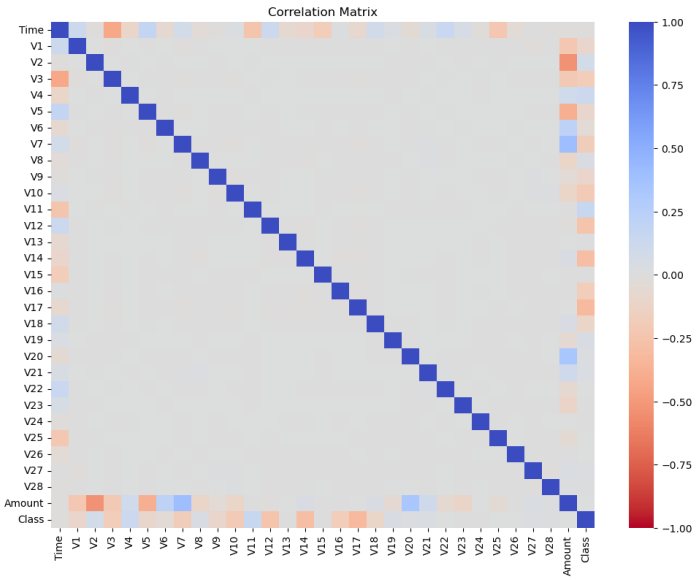


Fig. 2. Correlation matrix of the data

Consequently, as shown in figure 3 when PCA-transformed features are acquired (like 'V1', 'V2', 'V3', etc.), they are inherently ordered by the variance they represent from the original dataset, with 'V1' representing the most and the final component the least.

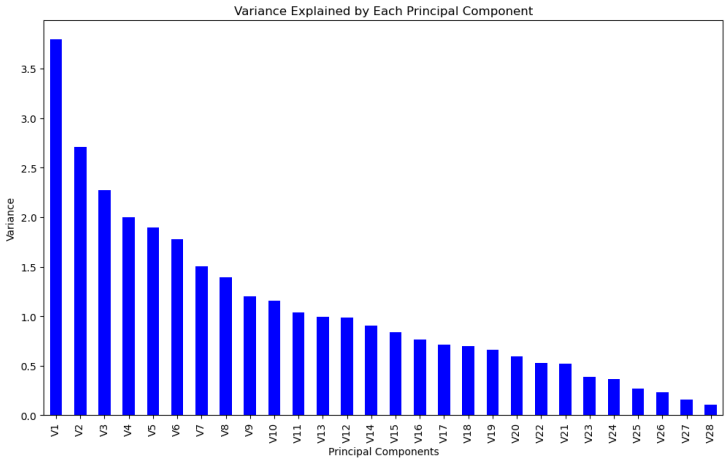


Fig. 3. Ordered variance of PCA data

Also, as is shown in figure 4, all features that were not applied to a PCA are not correlated with each other, which means that all data add some value to the problem.

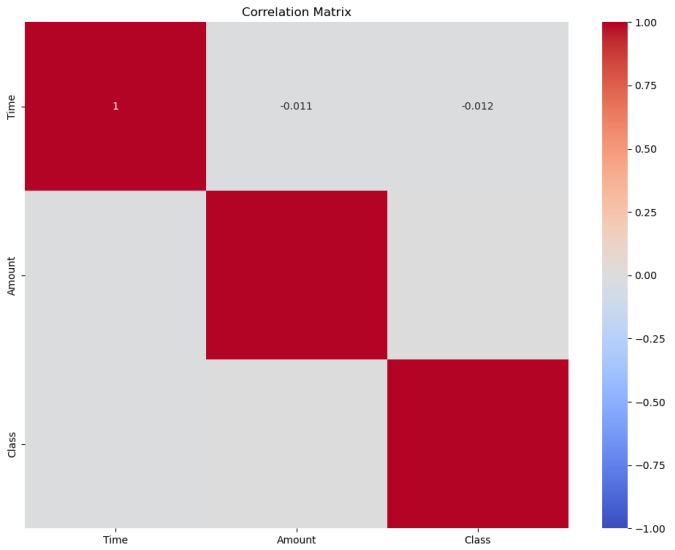


Fig. 4. Correlation matrix of the data for which PCA was not applied.

The bar charts displayed in figures 5 and 6 both negative and positive correlations between PCA-derived features and transaction amounts. Feature V5 exhibited the strongest negative correlation, suggesting an inverse relationship where higher values potentially correspond to lower transaction amounts. This could reflect particular transaction types or behaviors. Features V2 and V3 also showed negative correlations, albeit weaker than V5, while V23 demonstrated the mildest inverse relationship. On the positive side, Feature V7 was most strongly correlated with transaction amounts, implying it might capture elements tied to higher-value transactions. V20 also showed a significant positive correlation, followed by milder associations for V6 and V21.

The analysis extended beyond these features to include 'Class' and 'Time'. However, correlations with these dimen-

sions were found to be negligible, suggesting they do not have a pronounced linear relationship with transaction amounts within our dataset. This emphasizes the higher relevance of the PCA features in explaining transaction value patterns.

The identified correlations, both negative and positive, with PCA features are integral to our understanding of the data. They offer a foundation upon which to build and refine fraud detection models, ensuring that they are attuned to the most telling transaction characteristics.

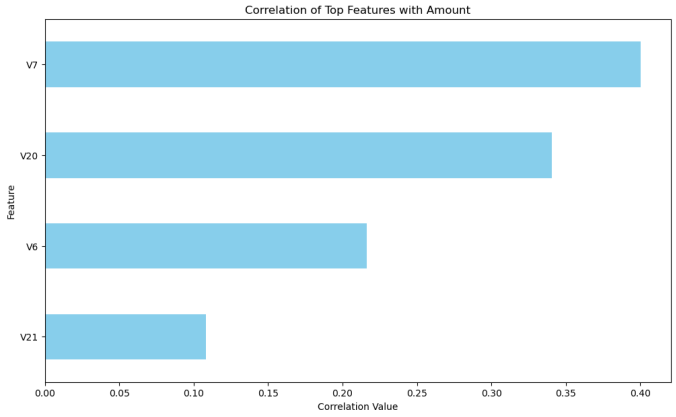


Fig. 5. Top negative values of the correlation matrix of the data for which PCA was not applied with PCA features.

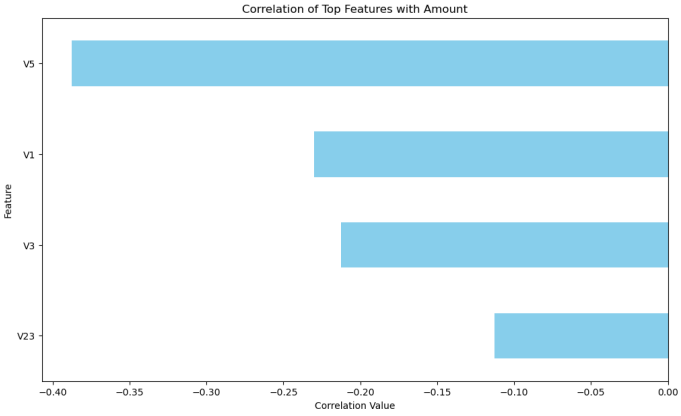


Fig. 6. Top negative values of the correlation matrix of the data for which PCA was not applied with PCA features.

F. Fraud vs Valid

The images 7 and 8 presents two boxplot visualizations comparing transaction amounts for valid and fraudulent transactions, clearly delineating the impact of outliers on the data representation.

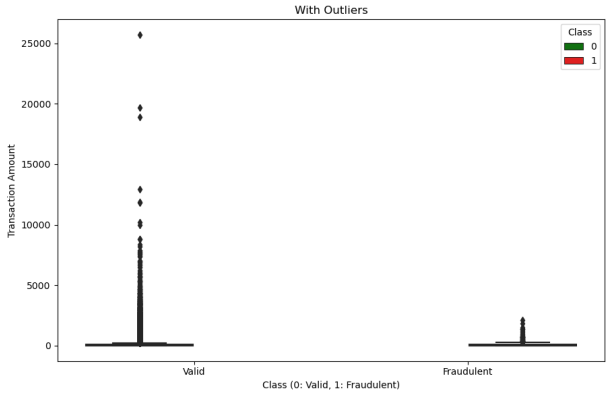


Fig. 7. Boxplot comparing valid transitions with fraud transactions.

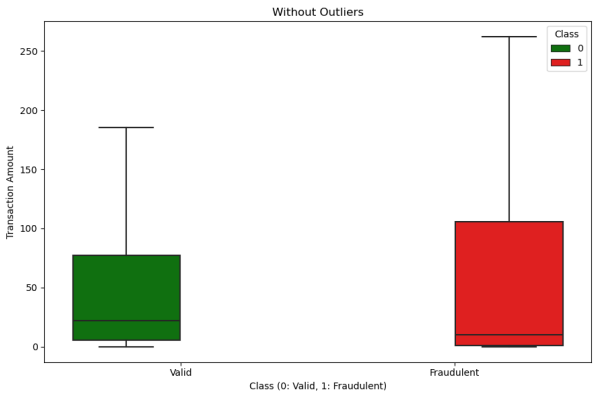


Fig. 8. Boxplot comparing valid transitions with fraud transactions.

In the first boxplot, titled ”With Outliers”, in figure 7 there is a contrast between valid and fraudulent transactions. For valid transactions, most data points are clustered near the bottom of the plot, indicating lower transaction amounts. However, several outliers extend significantly above the main cluster, suggesting a few valid transactions with unusually high amounts. In the case of fraudulent transactions, the range is much wider, and while the central tendency is higher than that of valid transactions, there are extreme outliers that reach substantially higher transaction amounts. These outliers in both categories suggest that while most transactions (both valid and fraudulent) are of lower amounts, there are exceptional cases with very high values.

The second boxplot, titled ”Without Outliers”, in figure 8 gives a more normalized view by excluding these outliers. The scale of the y-axis is dramatically reduced, allowing a clearer comparison of the typical transaction amounts between the two classes. Valid transactions exhibit a narrow interquartile range, reflecting a smaller variation in transaction amounts, and the median is close to the bottom of the box, implying that most valid transactions are of lower amounts. Fraudulent transactions, on the other hand, show a much larger interquartile range and a higher median, indicating that fraudulent transactions tend to involve higher amounts of money and have

a greater variance in transaction amounts compared to valid transactions.

I analyzed the distributions of transaction times and amounts with figure 9 to understand the temporal dynamics and monetary characteristics of the transactions.

The histogram of transaction times shows cyclical patterns, which correspond to daily transaction activity peaks. These peaks may reflect higher transaction volumes during specific hours of the day, suggesting that consumer behavior is influenced by typical daily routines.

The distribution of transaction amounts indicates that most transactions are of lower value, with a long tail stretching towards higher amounts, signifying the presence of outliers. To better understand the distribution's shape, we zoomed into the lower transaction values. The zoomed-in view reaffirms that a vast majority of transactions are small in amount, with a rapid decline in frequency as the transaction amount increases.

These distributions provide a foundational understanding of the dataset's characteristics. The time distribution can inform feature engineering for temporal patterns, while the amount distribution highlights the need for outlier handling in preprocessing steps. Both distributions will be crucial for the subsequent anomaly detection efforts.

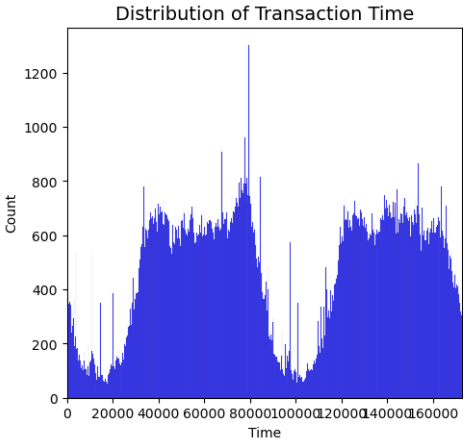


Fig. 9. Distribution of amount and transaction frequency over time.

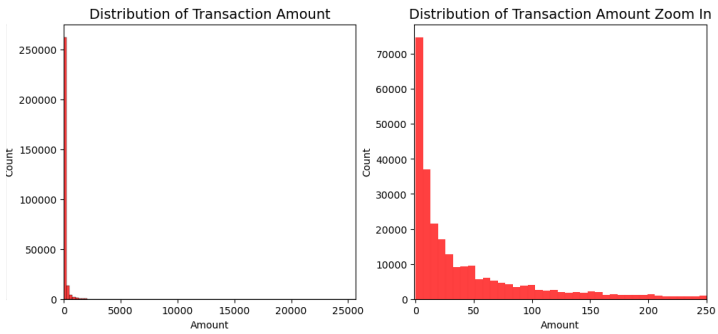


Fig. 10. Distribution of amount and transaction frequency over time.

The time density plot, in figure 10 for credit card transactions provides valuable insights into the temporal behavior of both valid and fraudulent transactions.

The plot 11 displays two distinct curves: one for valid transactions (in green) and another for fraudulent ones (in red). These curves represent the probability density of transactions occurring at different times, offering a clear visual comparison:

- The green curve (valid transactions) shows cyclical peaks, suggesting regular intervals of high transaction activity which may correspond to daily or weekly patterns in legitimate user behavior.
- The red curve (fraudulent transactions) also exhibits peaks but they do not coincide with the valid transaction peaks. This discrepancy could indicate preferential times for fraudulent activity, which may be off-peak hours when detection is less likely.
- The density of fraudulent transactions appears to have a more uniform distribution across time compared to valid transactions, suggesting that fraudulent activity does not follow the same temporal patterns as legitimate transactions.

Understanding these time-based patterns is crucial for developing time-sensitive fraud detection systems. The non-overlapping peaks suggest that incorporating time as a feature could improve the accuracy of such systems by targeting times when fraud is more likely to occur.

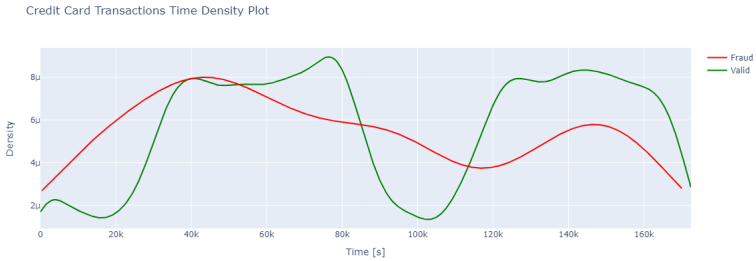


Fig. 11. Distribution of amount and transaction frequency over time.

1) Transactional Analysis Based on Hourly Activity: The primary analysis is based on the first histogram, which portrays the density of valid and fraudulent transactions across different hours of the day (Figure 12). The histogram reveals notable patterns:

- The distribution of valid transactions exhibits regular peaks and troughs, corresponding to the expected daily cycles of consumer activity.
- Fraudulent transactions, however, peak at different hours compared to valid transactions. Particularly, there is a significant peak in fraudulent activity in the early hours, a time when legitimate activity is typically lower.
- This contrast in patterns between valid and fraudulent transactions underscores potential temporal vulnerabilities that could be exploited by fraudsters.

The insights drawn from this plot are instrumental for enhancing fraud detection systems by adjusting monitoring

intensity according to the risk associated with different hours of the day.

Further detailed analyses of transaction patterns over 48-hour periods and the frequency of transactions by the hour of the day have been conducted. These additional studies corroborate the primary observations and provide an extended view of the transaction dynamics. The complete analyses are included in the annexes for reference.

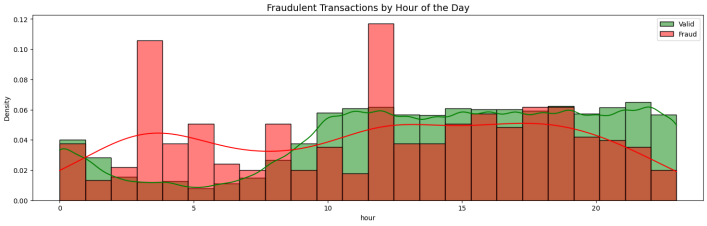


Fig. 12. 24 Hourly density of valid and fraudulent transactions highlighting the distinct patterns of activity.

The chart, in figure 13, shows the percentage of fraudulent transactions by hour over 24 hours, revealing distinct peaks that indicate that targeted hours for fraud are at dawn.

2) Analysis of Transaction Amount Frequencies for Valid and Fraudulent Transactions: The examination of transaction amount frequencies for both valid and fraudulent transactions offers a comparative look into their respective monetary behaviors.

The first histogram, figure 14, detailing the frequency of fraudulent transaction amounts, shows that the most common transactions occur in the lowest amount range, with a significant 36.38% falling into the $(0, 2]$ bracket. This is followed by a sharp drop to 9.33% in the next amount range of $(2, 4]$. The frequencies continue to decrease as the transaction amounts increase, which may indicate that while fraud is more frequent with lower amounts, it is not restricted to them.

Conversely, the histogram for valid transactions, figure 15, presents a different pattern. The most frequent valid transactions also occur in the lowest amount range, yet only comprise 16.31%, which is less than half of the frequency for fraudulent transactions in the same range. Subsequent amount ranges show a more gradual decrease in frequencies compared to fraudulent transactions.

The contrast in the frequency distributions between valid and fraudulent transactions suggests distinct operational patterns. The concentration of fraudulent transactions in lower amount ranges could be a strategic choice by fraudsters to avoid detection, as smaller transactions might be less likely to raise suspicion.

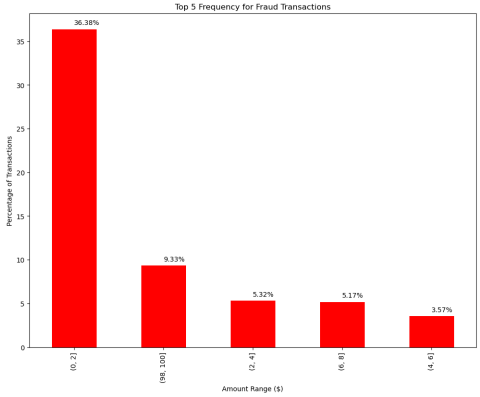


Fig. 14. Percentage frequencies of fraudulent transactions by amount range.

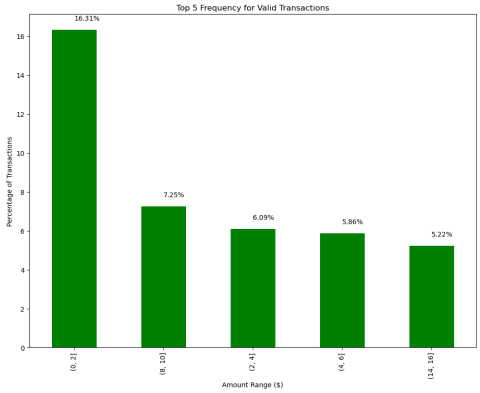


Fig. 15. Percentage frequencies of valid transactions by amount range.

3) Hourly Transaction Statistics for Valid and Fraudulent Activities: The transaction data was analyzed to extract hourly statistics, comparing the average transaction values and the range between the minimum and maximum transaction amounts for both valid and fraudulent transactions.

The statistics for valid transactions indicate a relatively stable mean transaction value throughout most hours of the day, with a notable peak during the late evening. The min-max range is consistently wide, suggesting a diverse set of transaction values at all hours.

Fraudulent transactions, however, show more variability in the mean transaction value, with several distinct peaks observed, possibly indicating targeted times for fraudulent activities. The min-max range for these transactions is also varied but generally narrower than for valid transactions, which may reflect the focused nature of fraudulent behavior.

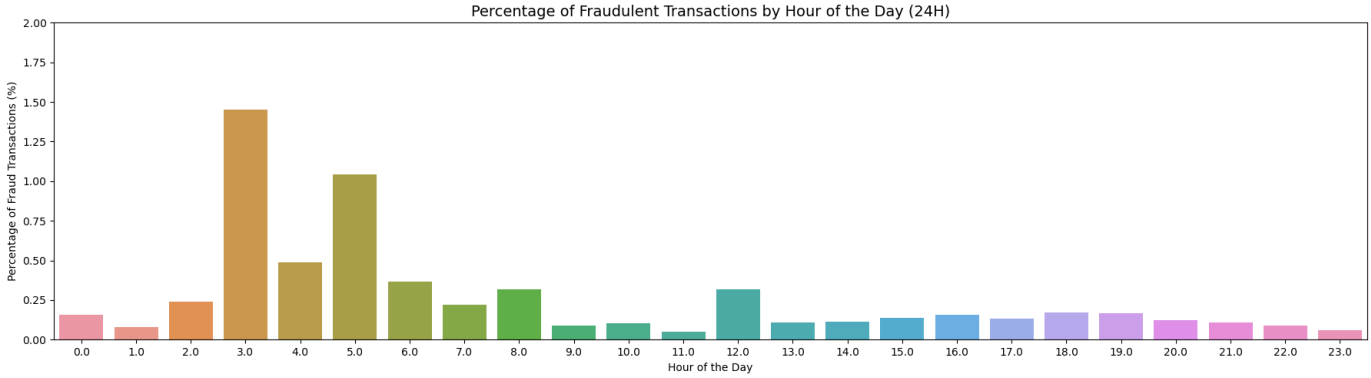


Fig. 13. Fraud by hour.

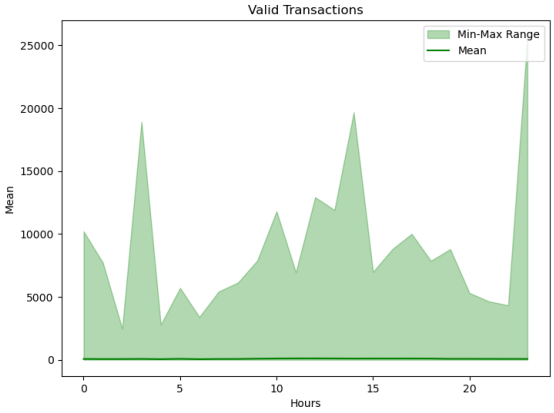


Fig. 16. Hourly transaction statistics showing the mean and min-max range for valid transactions.

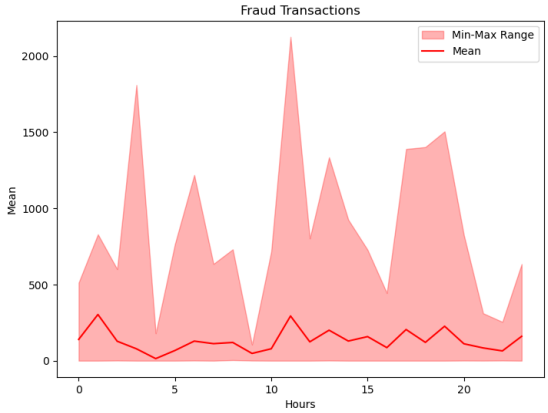


Fig. 17. Hourly transaction statistics showing the mean and min-max range for fraudulent transactions.

4) Distribution After Logarithmic Transformation: The histogram on the left, in figure 18, illustrates the distribution of transaction amounts. It shows a highly right-skewed distribution with a steep drop-off, indicating that lower transaction amounts are far more common than higher ones.

The histogram on the right, in figure 19, applies a logarithmic transformation to the transaction amounts, using a base of 10. This transformation reveals a more nuanced view of the data, with multiple peaks that were not observable in the original distribution. Applying a logarithmic transformation is particularly useful in data analysis when dealing with variables that exhibit a highly skewed distribution, as is often the case with financial transaction amounts. The log transformation helps to stabilize the variance across the data, making the distribution more symmetric and, in many cases, more closely resembling a normal distribution.

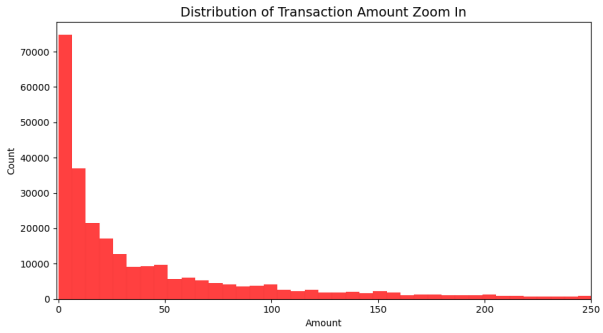


Fig. 18. Transaction amount distribution before applying a logarithmic transformation.

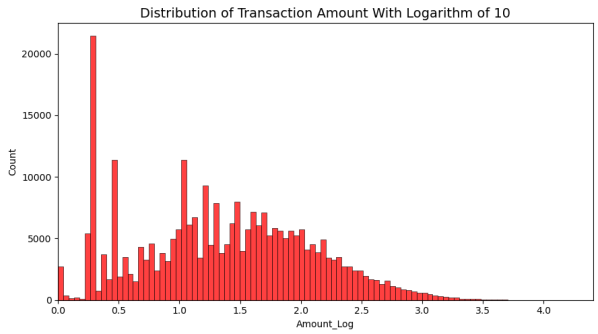


Fig. 19. Transaction amount distribution after applying a logarithmic transformation.

5) Radial Analysis of Fraudulent Transactions: The radial plot visualizes the percentage of fraudulent transactions by hour, showcasing pronounced spikes during the early hours, particularly around 3 am. This pattern suggests a higher incidence of fraudulent activity during off-peak hours, which is likely to evade detection.

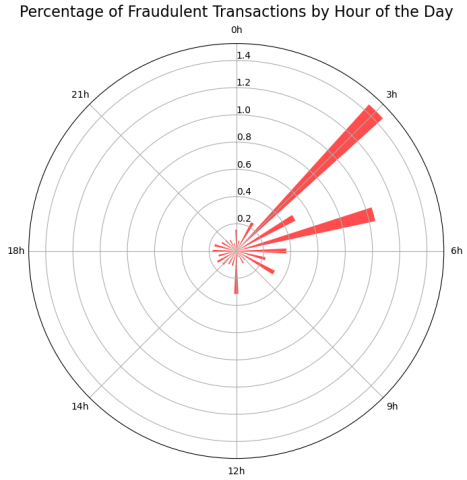


Fig. 20. Radial Analysis of Fraudulent Transactions

G. Outliers overview

This boxplot in figure 21 illustrates the distribution of transaction amounts after a logarithmic transformation. The transformation normalizes the distribution, reducing the skewness significantly. However, it also highlights the presence of outliers, as evidenced by the points beyond the whiskers of the boxplot. These outliers suggest transaction amounts that are significantly higher than the typical range, even after the log transformation.

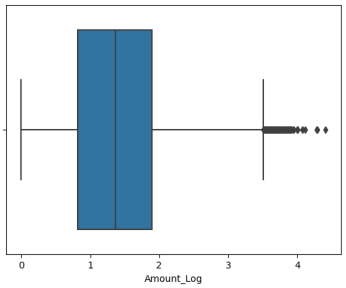


Fig. 21. Boxplot of logarithmic transaction amounts.

The bar chart represents the count of outliers within the class labeled '0'. These outliers are transactions that fall outside the typical range when considering a logarithmic transformation of transaction amounts. The chart indicates a substantial number of outliers in this class, which could correspond to legitimate transactions with unusually high amounts.



Fig. 22. Analysis of Outliers by Class.

1) t-SNE: t-Distributed Stochastic Neighbor Embedding (t-SNE) is a dimensionality reduction algorithm designed for visualizing high-dimensional datasets. The resulting t-SNE plot, illustrated in Figure 23, provides a two-dimensional representation that preserves local structures, effectively revealing underlying patterns in the data.

In the scatter plot, each data point is color-coded to represent distinct classes or characteristics. Clusters of green points denote regions of high data density, indicating inherent groupings within the dataset corresponding to different categories or behaviors. Conversely, red points, distinctly separated from green clusters, may signify outliers or a minority class, revealing anomalies or unique data points.

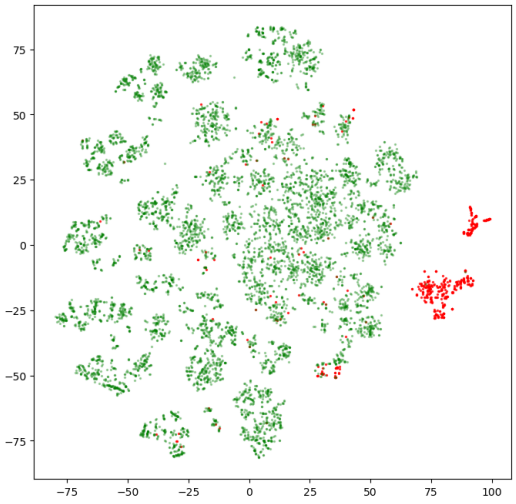


Fig. 23. t-SNE visualization of outliers.

III. Data Preprocessing

Data transformation is a critical step in the preprocessing phase of a machine learning pipeline. It involves converting raw data into a format that is more suitable for modeling. This process not only simplifies the subsequent analysis and helps in improving the accuracy of the model but also ensures better feature representation.

A. Duplication Removal

The first step in our data preprocessing is the removal of duplicate entries. Duplicate data can lead to biased results as it over-represents certain information. Removing duplicates ensures that each data point is unique, thereby contributing to the robustness of the machine-learning model.

B. Feature Engineering

Feature engineering is a crucial step in which we transform or create new features that can significantly improve the performance of the model. In our case, we focus on time-based feature engineering. We convert the 'Time' column into hourly features, which help in capturing patterns across different hours of the day. This transformation aids in understanding the periodic trends and anomalies in the data.

C. Logarithmic Transformation

Logarithmic transformation is applied to the 'Amount' column. This transformation is particularly useful in dealing with skewed data and can help stabilize variance, making patterns more visible and data more interpretable. By applying a logarithm of 10, we reduce the impact of large outliers and achieve a more normalized scale.

D. Splitting Data into Features and Targets

I divided data into distinct sets: training, testing, and validation. This separation is important for both the learning phase and the assessment of the model's performance.

In the specific context of our data, we have used the concept of time, represented by 'hour48', to separate our data chronologically. The first 24 hours of data are used for training, allowing the model to learn from the earliest data points. The subsequent data points are used for testing, enabling us to evaluate the model's predictive power on subsequent events.

Further, we refine our testing set to carve out a validation set, which is 15% of the test data. This small chunk is randomly selected to represent various points in time, ensuring that we have a comprehensive set that maintains the natural order of events. The test data is 85% of the second day for the final evaluation of the model's performance.

We could do that because the transactions were equilibrated, as we can see in figure 24. Frauds are equilibrated as well.

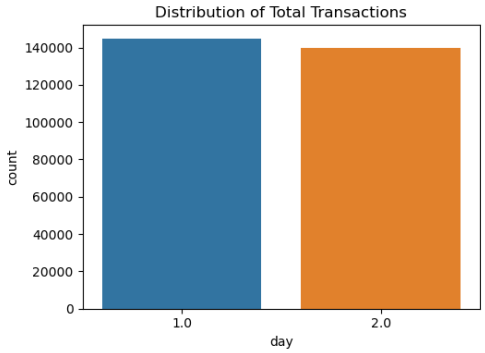


Fig. 24. Distribution by day.

E. Data Normalization

Finally, we normalize the data using StandardScaler, which scales the features to have a mean of 0 and a standard deviation of 1. This step is crucial for algorithms that are sensitive to the scale of the data, such as linear models and k-nearest neighbors.

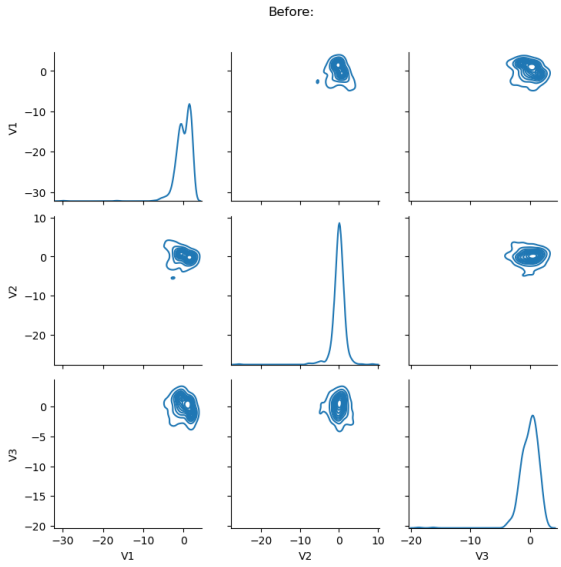


Fig. 25. Data before normalization.

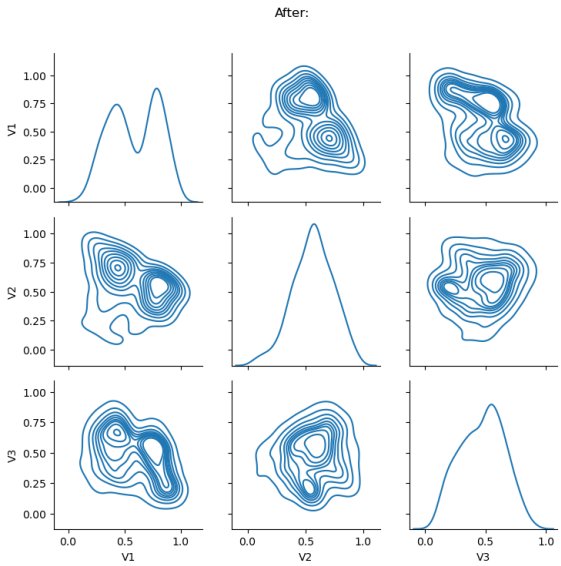


Fig. 26. Data after normalization.

F. Resampling

Techniques like SMOTE (Synthetic Minority Over-sampling Technique) and Near Miss are employed to address the unbalanced issue by balancing the class distribution. While SMOTE focuses on oversampling the minority class, Near Miss is geared towards undersampling the majority class.

1) SMOTE (Synthetic Minority Over-sampling Technique): SMOTE is an advanced oversampling technique that generates synthetic samples for the minority class. This technique is used to avoid the overfitting problem commonly encountered with random oversampling [2].

SMOTE operates by randomly picking a point from the minority class and computing the k-nearest neighbors for this point. The synthetic points are then added between the chosen point and its neighbors. This process creates a more general and broader decision region for the minority class. Enhances the decision space, allowing models to learn more nuanced patterns.

Although is less effective in high-dimensional spaces and can amplify noise by creating synthetic samples from noisy data.

2) Near Miss: Near Miss is an undersampling technique that aims to balance class distribution by reducing the size of the majority class. It selects samples based on their distance from the minority class [5].

Several versions of Near Miss exist, but they generally involve selecting majority-class samples that are closest to the minority class in the feature space. This selection is done with the shortest average distance to the nearest minority class samples.

Helps to balance class distribution, potentially improving model performance, but there is a risk of losing important information from the majority class.

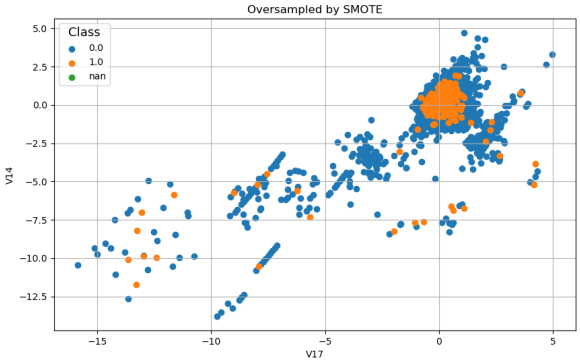


Fig. 27.

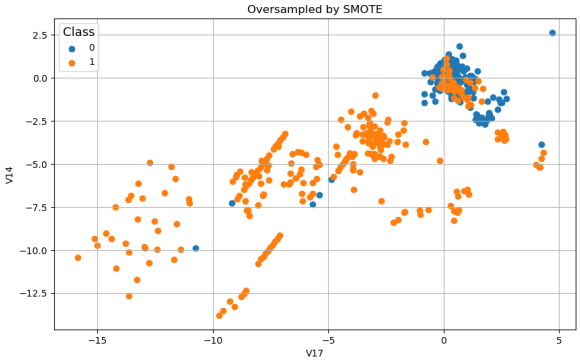


Fig. 28.

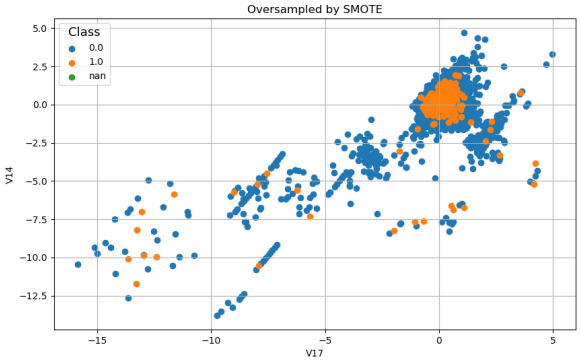


Fig. 29.

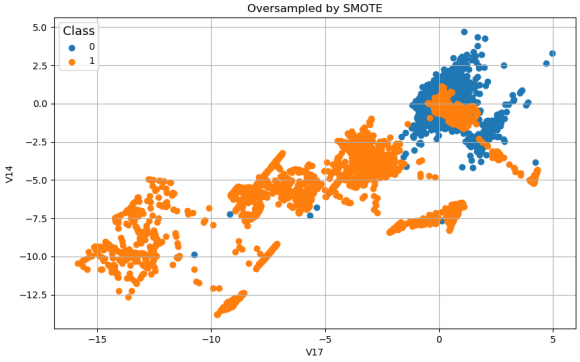


Fig. 30.

G. Metrics used

F1 Score Explanation: The F1 score is a measure of a model's accuracy in a binary classification task, which considers both precision and recall of the test to compute the score. The F1 score can be understood as a weighted average (harmonic mean) of the precision and recall, where an F1 score reaches its best value at 1 (perfect precision and recall) and worst at 0.

Key Concepts

Precision: This is the number of true positive results divided by the number of all positive results, including those not identified correctly. It reflects the model's ability to identify only the relevant objects.

$$Precision = \frac{TP}{TP + FP}$$

Recall (Sensitivity or True Positive Rate): This is the number of true positive results divided by the number of all samples that should have been identified as positive. It reflects the ability of the model to find all the relevant cases within a dataset.

$$Recall = \frac{TP}{TP + FN}$$

F1 Score: This is the harmonic mean of precision and recall. The harmonic mean is used rather than the arithmetic mean as it punishes extreme values more. A model with perfect precision and recall will have an F1 score of 1. A model with a perfect precision but poor recall, or vice versa, will have an F1 score closer to 0.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Alternatively:

$$F1 = \frac{2TP}{2TP + FP + FN}$$

When to Use F1 Score

F1 Score is useful when seeking a balance between precision and recall, especially if there is an uneven class distribution.

Evaluating Classifier Performance

The Receiver Operating Characteristic (ROC) curve and the F1 Score are pivotal in classifier evaluation. The ROC curve, which plots the True Positive Rate against the False Positive Rate at various thresholds, assesses the classifier's ability to discriminate between classes without fixing a threshold. Its Area Under the Curve (AUC) provides an aggregate measure of performance, with 1.0 representing perfect discrimination.

While the ROC curve offers a threshold-independent evaluation, the F1 Score gives a precise measure at a chosen threshold (0.5 in this case), encapsulating the trade-off between false positives and false negatives essential for real-world applications.

IV. Models

A. SVM

Support Vector Machines (SVM) are a set of supervised learning methods used for classification, regression, and outliers detection. The SVM algorithm is particularly well-suited for binary classification tasks [3].

1) SVM for Binary Classification: In binary classification, the goal of the SVM algorithm is to find the optimal hyperplane that separates two classes of data points with the maximum margin. The hyperplane is defined as the decision boundary that maximizes the distance between the closest points of the two classes, which are known as support vectors.

Mathematically, if we have a dataset with features $\mathbf{x}$ and labels $y \in \{+1, -1\}$, the SVM finds the optimal hyperplane by solving the following optimization problem:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \tag{1}$$

$$\text{subject to } y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1, \ \forall i \tag{2}$$

Here, $\mathbf{w}$ represents the weights, b is the bias term, and the dot product $\mathbf{w} \cdot \mathbf{x}_i$ signifies the decision function. The constraint ensures that all data points are correctly classified with a margin at least 1.

2) Kernel Trick: One of the key features of SVM is the use of kernels, which allows the algorithm to handle non-linearly separable data. The kernel trick involves mapping the input features into a higher-dimensional space where a linear separation is possible.

B. One Class SVM

One-class SVM is an unsupervised learning algorithm that is typically used for anomaly detection. Unlike the traditional SVM that is used for binary classification tasks, One-Class SVM is designed to detect the outliers in a dataset by learning a decision function for novelty detection: it classifies new data as similar or different to the training set.

The main idea of One-Class SVM is to fit a decision boundary around the normal data points. It works by mapping the input data into a high-dimensional feature space using a kernel function and then finding the maximum margin hyperplane that best separates the data from the origin.

Given a set of training data, the One-Class SVM solves the following optimization problem:

$$\min_{\mathbf{w},\xi_i,\rho} \frac{1}{2}\|\mathbf{w}\|^2 + \frac{1}{\nu n}\sum_{i=1}^{n}\xi_i - \rho \tag{3}$$

$$\text{subject to } (\mathbf{w} \cdot \Phi(\mathbf{x}_i)) \geq \rho - \xi_i, \ \xi_i \geq 0, \ \forall i \tag{4}$$

Here, $\mathbf{w}$ is the weight vector, Φ denotes the feature map corresponding to the kernel, ξ_i are the slack variables, ρ is the decision threshold, ν is a parameter that controls the trade-off between maximizing the distance of the hyperplane from the origin and the number of data points that are allowed to be outliers (fall on the other side of the hyperplane), and n is the number of training examples.

C. Random Forrest

Random Forest is a versatile machine-learning algorithm capable of performing both regression and classification tasks. It is a type of ensemble learning method, where a group of weak models combine to form a strong model. In the case of Random Forest, the weak models are decision trees [4].

1) Random Forest Algorithm: Random Forest builds multiple decision trees and merges them together to get a more accurate and stable prediction. The fundamental concept behind Random Forest is simple but powerful: combine the predictions of several trees, which are individually weak and prone to overfitting, to produce a robust, generalizable model.

The algorithm works as follows:

- For every tree, a random sample of the training dataset is selected.
- At each node in the tree, a subset of features is chosen at random to decide the split.
- Each tree is grown to the largest extent possible, and no pruning is performed.
- The final prediction is made by averaging the predictions (regression) or taking a majority vote (classification) of all the individual trees.

Random Forest can handle binary features, categorical features, and numerical features without the need for scaling. Also are less likely to overfit than individual decision trees capturing complex interaction structures between features making the robust.

D. Isolation Forrest

Isolation Forest is an unsupervised learning algorithm for anomaly detection. It isolates anomalies instead of profiling normal data points and is particularly efficient for large datasets.

1) Isolation Forest Algorithm: The Isolation Forest algorithm isolates observations by randomly selecting a feature and then randomly selecting a split value between the maximum and minimum values of the selected feature. The key idea is that anomalies are few and different, and therefore they are easier to isolate compared to normal points.

The algorithm works as follows:

- A random subset of the data is used to create a tree (an 'isolation tree'). Random partitioning is used to recursively split the data into nodes of the tree.
- The number of splits required to isolate a sample is equivalent to the path length from the root node to the terminating node.
- This path length, averaged over many trees, becomes a measure of normality and our decision function.
- Anomalies are those instances for which the average path length is short.

Isolation Forests have a low linear time complexity and a small memory requirement.

E. Feedforward Neural Network

A Feedforward Neural Network is a type of artificial neural network where connections between the units do not form a cycle. This network is also known as a Multilayer Perceptron (MLP).

In its simplest form, a feedforward network consists of three types of layers: an input layer, one or more hidden layers, and an output layer. The sequential model in neural networks implies constructing these layers in a linear sequence.

- The **input layer** is responsible for receiving the signals and passing them forward.
- The **hidden layers** perform nonlinear transformations of the inputs entered through the input layer.
- The **output layer** is the final layer that provides the output for the network.

The key feature of a feedforward network is that data moves in one direction. There is no back-looping of the data as in recurrent neural networks.

1) Learning in Feedforward Neural Networks: Learning in feedforward neural networks typically involves backpropagation, where the output error is propagated backwards through the network to update the weights, and gradient descent, or one of its variants, to minimize the cost function.

F. Autoencoder

Autoencoders are neural networks designed for the specific task of data encoding, commonly used for dimensionality reduction or feature extraction. The primary objective of an autoencoder is to reconstruct its input as accurately as possible, which encourages the network to learn a compact and informative representation of the data [1].

Encoder: The encoder segment of an autoencoder compresses the input into a lower-dimensional latent space, capturing the essence of the data. It functions by mapping the high-dimensional data to a lower-dimensional latent representation, learning to preserve as much of the significant information as possible.

Decoder: Following the encoding process, the decoder attempts to reverse the transformation applied by the encoder, reconstructing the data from the compressed code back to its original dimensionality. The goal is to generate an output that closely resembles the original input data, effectively undoing the encoding operation.

Loss Function: To measure the performance of an autoencoder, a loss function is employed to assess the difference between the original input and the output produced by the decoder. The most common loss function is the mean squared error, which quantifies the discrepancy between the reconstructed data and the original data. The training process involves minimizing this error, which guides the network towards a more accurate reconstruction of the inputs.

Autoencoder for Fraud Detection: The concept of using an autoencoder for fraud detection is predicated on its bottleneck architecture, which compels the network to learn a condensed representation of the input data. By training exclusively on normal transactions, the autoencoder masters the task of reproducing these regular patterns with high fidelity. Conversely, if a transaction significantly deviates from the norm—such as a fraudulent one—the autoencoder struggles to accurately reconstruct it, resulting in a high reconstruction loss. Transactions yielding a loss above a predetermined threshold are then flagged as anomalous, indicating potential fraud. In my case, the detection of fraud was based on a modified Z-score method using the Median Absolute Deviation (MAD), where data points with a score greater than 6 were classified as

outliers or potential fraud. However, for ROC/AUC, we do not need to fix the threshold.

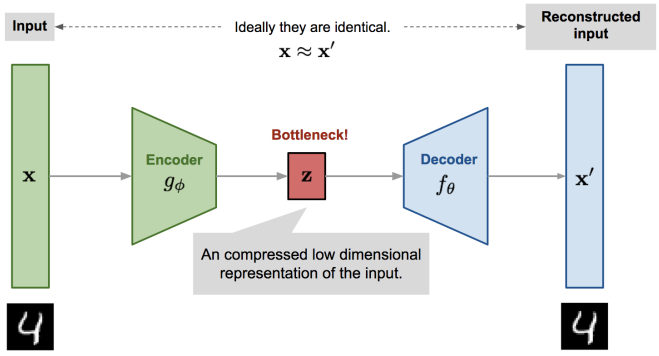


Fig. 31. Schematic representation of an autoencoder used for fraud detection.

Despite their utility, autoencoders, if the latent space is too large, the autoencoder may learn to copy the input to the output without learning useful features. Conversely, if it is too small, important information may be lost during encoding.

Hyperparameter Tuning with Grid Search

For optimizing the models' hyperparameters, a grid search approach was employed. This method systematically works through multiple combinations of parameter tunes, cross-validating as it goes to determine which tune gives the best performance. The following table outlines the hyperparameter space explored for each model.

Model	Parameters
RFC	Max depth: [None, 4, 6, 8], Min samples split: [2, 5, 10], Max features: [auto, sqrt, log2]
SVC	C: [0.1, 0.2, 1, 10], Gamma: [scale, auto], Degree: [3, 4, 5], Kernel: [linear, rbf, poly]
OneClass SVM	Gamma: [scale, auto], Kernel: [linear, rbf, poly, sigmoid]
Isolation Forest	Estimators: [2, 50, 100], Contamination: [0.1, 0.2, 0.3], Max features: [1.0, 0.5, 0.1], Bootstrap: [True, False]
NN	Layers: Dense (32-256, relu), Dropout (0-0.5), Output: Dense (1, sigmoid), Optimizer: adam/sgd, Loss: binary_crossentropy
Autoencoder	Layers: Dense (Input dim, elu) x5, Dense (4, elu) x4

TABLE II
PARAMETERS AND HYPERPARAMETERS OF MACHINE LEARNING MODELS

Upon completion of the grid search, the best-performing model configuration for each algorithm was identified based on the cross-validated performance metrics. These optimal models, characterized by their respective hyperparameter settings as detailed in Table II, were then preserved for subsequent application on the test dataset. This approach ensured that the models were not only tuned for maximum efficiency but also validated against unseen data, thereby enhancing the reliability and robustness of our predictive analysis.

The results were chosen considering each type of oversampling and the best combination is not the focus of this work, so will not be shown.

V. RESULTS

Model Performance Evaluation using SMOTE

The performance of various machine learning models was compared using a dataset augmented with 30% Synthetic Minority Over-sampling Technique (SMOTE) to address class imbalance. The metrics selected for this comparison were ROC AUC (Receiver Operating Characteristic Area Under the Curve), F1 Score, and Accuracy, each reflecting different aspects of model performance.

Accuracy measures the proportion of true results (both true positives and true negatives) among the total number of cases examined. While it gives a quick snapshot of model performance, accuracy can be misleading in the presence of class imbalance, which is mitigated in our case by the use of SMOTE.

The RandomForestClassifier and SVC (Support Vector Classifier) models exhibited superior performance across all metrics, indicating robust predictive capabilities. OneClassSVM and IsolationForest, while generally used for outlier detection rather than classification, showed lower performance in this context. The Neural Network demonstrated a high accuracy but a moderate F1 Score, suggesting some room for improvement in the balance between precision and recall. Lastly, the Autoencoder, typically used for dimensionality reduction and feature learning, showed promising ROC AUC but a lower F1 Score and Accuracy, which could be attributed to its unsupervised nature and the complexity of reconstructing SMOTE-augmented data.

These results underscore the importance of selecting the right metrics for model evaluation, particularly in scenarios involving imbalanced datasets. The choice of metric should align with the specific objectives and the cost implications associated with the predictive modeling task at hand.

Model	ROC AUC	F1 Score	Accuracy
RandomForestClassifier	0.9177	0.5778	0.9985
SVC	0.9404	0.7137	0.9990
OneClassSVM	0.7255	0.0055	0.4755
IsolationForest	0.6567	0.0574	0.9839
Neural Network	0.9283	0.4125	0.9967
Autoencoder	0.8863	0.0691	0.9677

TABLE III
MODEL PERFORMANCE COMPARISON USING 30% SMOTE.

ROC Curve Analysis of SVC Model: The Receiver Operating Characteristic (ROC) curve of the Support Vector Classifier (SVC) model, as shown in Figure 32, demonstrates a high Area Under the Curve (AUC) of 0.94. This high AUC value indicates the strong ability of the SVC model to distinguish between the positive class (fraudulent transactions) and the negative class (legitimate transactions). The curve closely hugs the top left corner of the plot, signifying a high true positive rate and a low false positive rate, which is ideal in a fraud detection context where the cost of misclassifying a legitimate transaction as fraudulent (false positive) is high. The nearly vertical rise and sustained plateau of the curve reflect the

model's effectiveness in classifying the transactions accurately across different thresholds.

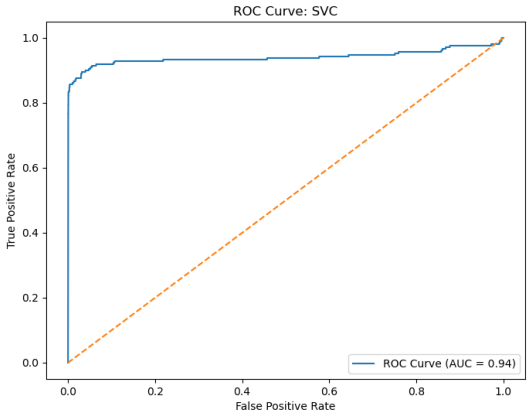


Fig. 32. ROC Curve of the SVC model displaying a high AUC, indicative of excellent model performance.

Confusion Matrix Analysis of Anomaly Detection Models: The performance of the OneClassSVM and IsolationForest models, two common approaches for anomaly detection, can be evaluated through their confusion matrices, depicted in Figures 33 and 34 respectively.

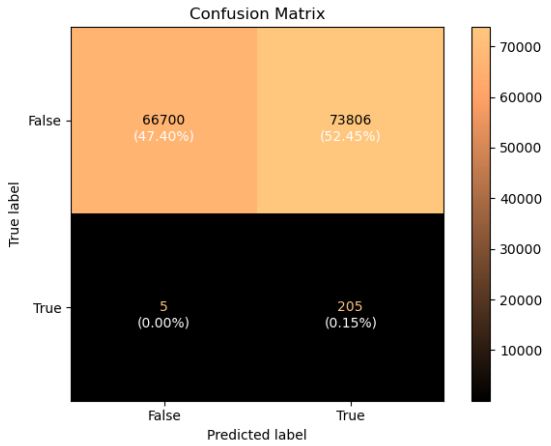


Fig. 33. Confusion matrix of the OneClassSVM model.

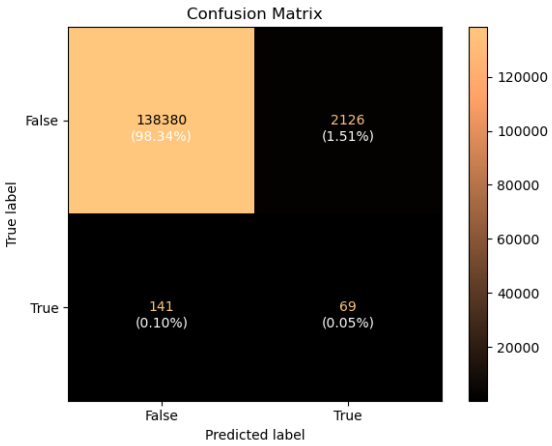


Fig. 34. Confusion matrix of the IsolationForest model.

For the OneClassSVM model, the matrix illustrates a considerable amount of false positive predictions, indicating the model's ability to correctly identify the majority of non-outlier transactions. However, the true positive rate is notably low, reflecting challenges in accurately detecting the rarer outlier events.

Conversely, IsolationForest's confusion matrix shows a slightly better detection rate true negatives but at the cost of a a low detection of fraud.

In both cases, the trade-off between detecting true positives and avoiding false positives is apparent and highlights the intricate balance anomaly detection models must achieve. These visual representations underscore the importance of tuning model parameters to optimize for the specific needs of the fraud detection task, balancing the sensitivity to anomalies with the practicality of maintaining a manageable false positive rate.

Comparative ROC AUC Score Analysis: A visual comparison of the models' ROC AUC scores, shown in Figure 35, indicates the relative ability of each model to distinguish between classes. Higher scores are indicative of better model performance concerning correctly identifying true positives while minimizing false positives.

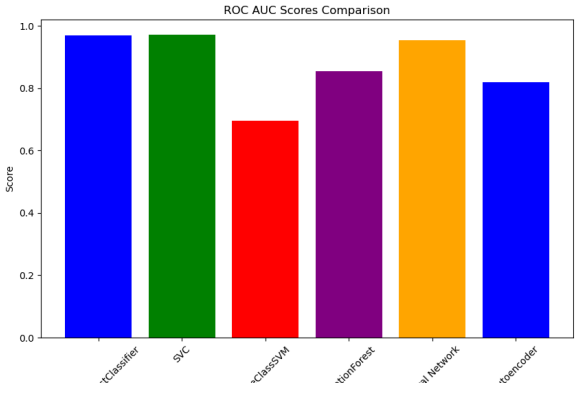


Fig. 35. Bar chart comparison of ROC AUC scores across various models.

The bar chart illustrates that while all models perform reasonably well, the SVC and Neural Network exhibit particularly high ROC AUC scores, suggesting their strong discriminative abilities in this scenario. The OneClassSVM and IsolationForest, while still above the baseline, show lower AUC values, which may indicate a reduced capability in handling the classification task as effectively as the other models.

1) Comparative Analysis of Model Performance with NearMiss: In our endeavor to develop a robust model for fraud detection, we have employed NearMiss, an undersampling technique, to address the class imbalance inherent in our dataset. This method has been instrumental in equalizing the distribution of classes, thus allowing for a more meaningful comparison of model performance. Our analysis has been grounded on a comprehensive evaluation using three pivotal metrics: ROC AUC, F1 Score, and Accuracy.

The RandomForestClassifier showcased a commendable ROC AUC score, reflecting its aptitude for class differentiation. Nevertheless, the model's F1 Score and Accuracy did not quite match the expectation set by the ROC AUC, suggesting that while the model is good at distinguishing classes, it may be less effective when it comes to the precision and recall balance, particularly after NearMiss undersampling.

In stark contrast, the SVC emerged as a top performer, excelling across all three metrics. The model's superior ROC AUC score indicates its exceptional ability to discriminate between fraudulent and legitimate transactions. Moreover, the high F1 Score of the SVC points to an effective balance between precision and recall, reinforcing its suitability for our fraud detection framework.

The OneClassSVM and IsolationForest models, typically preferred for anomaly detection, recorded lower scores on all fronts. Their modest ROC AUC values and especially their F1 Scores, which were significantly impacted by the NearMiss technique, signal limitations that warrant further investigation.

Conversely, the Neural Network and Autoencoder adapted more favorably to the class-balanced dataset, as suggested by their ROC AUC scores. Despite the unsupervised nature of the Autoencoder, it achieved a laudable ROC AUC score. However, both models demonstrated room for improvement in their F1 Scores, which is crucial for maintaining a balance between identifying true fraud cases and minimizing false alerts.

These insights underscore the criticality of model selection in accordance with the dataset's nuances and the overarching goals of the fraud detection system. Given the results, the SVC's robust performance across the board suggests that it may be the most appropriate choice for this task, particularly when the focus is on striking a balance between detecting fraud accurately and minimizing false positives.

Model	ROC AUC	F1 Score	Accuracy
RandomForestClassifier	0.8748	0.0151	0.8644
SVC	0.9615	0.4333	0.9970
OneClassSVM	0.4820	0.0028	0.3880
IsolationForest	0.5639	0.0046	0.7748
Neural Network	0.8942	0.0504	0.9585
Autoencoder	0.8993	0.0758	0.9699

TABLE IV
COMPARISON OF MODEL PERFORMANCE METRICS USING NEARMISS.

Analysis of Model Performance with 15% SMOTE: Our comprehensive analysis, as summarized in Table V, presents the data balanced using 15% SMOTE. This technique was employed to mitigate the challenges posed by the imbalanced nature of the dataset, particularly in the context of fraud detection.

The RandomForestClassifier exhibited an impressive ROC AUC of 0.9308, coupled with a high F1 Score of 0.6967, indicating its strong capability in both class separation and maintaining a balance between precision and recall. Its remarkable accuracy further reinforces its efficacy in correctly identifying fraudulent transactions.

The SVC model achieved the highest ROC AUC of 0.9605 among all models, signifying its superior discriminative power. However, its F1 Score of 0.4530, while respectable, suggests some limitations in terms of precision-recall balance compared to the RandomForestClassifier.

Contrastingly, the OneClassSVM showed a significantly lower performance across all metrics, particularly in its F1 Score and Accuracy. This underperformance could be attributed to its inherent characteristics being less suited for this specific dataset, especially after SMOTE application.

IsolationForest and Neural Network models displayed moderate performances, with the Neural Network showing a promising ROC AUC of 0.9393. However, both models had relatively lower F1 Scores, indicating potential areas for improvement in precision and recall balance.

The Autoencoder, despite not being the top performer, showed a decent ROC AUC of 0.8846. Its low F1 Score of 0.0714, however, highlights the challenges faced in anomaly detection tasks, particularly in datasets augmented with synthetic samples through SMOTE.

Model	ROC AUC	F1 Score	Accuracy
RandomForestClassifier	0.9308	0.6967	0.9991
SVC	0.9605	0.4530	0.9973
OneClassSVM	0.6941	0.0049	0.4033
IsolationForest	0.7922	0.0499	0.9648
Neural Network	0.9393	0.4547	0.9971
Autoencoder	0.8846	0.0714	0.9690

TABLE V
COMPARISON OF MODEL PERFORMANCE METRICS USING 15% SMOTE FOR DATA BALANCING.

Comparative Analysis of Model Performances Across Different Balancing Techniques: In our comprehensive assessment, we compared the performances of several machine learning models using three different data balancing techniques: 30% SMOTE, NearMiss, and 15% SMOTE. The comparison, as detailed in Tables III, IV, and V, provides valuable insights

into how each model adapts to different data preprocessing strategies.

a) Impact of 30% SMOTE (Table III):: Using 30% SMOTE, the SVC and RandomForestClassifier showed excellent ROC AUC scores, with SVC leading in terms of F1 Score. This suggests that a moderate level of SMOTE helps in achieving a good balance between precision and recall, especially for complex models like SVC.

b) Impact of NearMiss (Table IV):: The application of NearMiss significantly altered the model performances. While SVC continued to exhibit high ROC AUC, its F1 Score decreased compared to the 30% SMOTE scenario. This indicates that while SVC remains robust in class separation, its precision-recall balance is affected by the NearMiss technique.

c) Impact of 15% SMOTE (Table V):: Reducing the SMOTE to 15% presented an interesting shift in model performances. The RandomForestClassifier displayed a notable increase in F1 Score, suggesting that a lower level of SMOTE might be more suitable for certain models. However, the overall trend remains consistent, with SVC showing strong ROC AUC and relatively high F1 Scores.

d) Overall Observations:: Across all methods, models like OneClassSVM and IsolationForest generally underperformed in terms of F1 Score and Accuracy, which might be due to their inherent suitability for different types of data distribution. The Neural Network and Autoencoder showed varied responses to the balancing techniques, indicating that their performance is sensitive to the nature of the data preprocessing.

Selection of Ensemble Methods Based on Outlier Detection: In the process of selecting an optimal ensemble method, visualizations such as scatter plots and Venn diagrams prove to be invaluable. Figure 36 shows a scatter plot of detected outliers, providing an intuitive understanding of the spread and overlap of normal and anomalous data points as identified by the models.

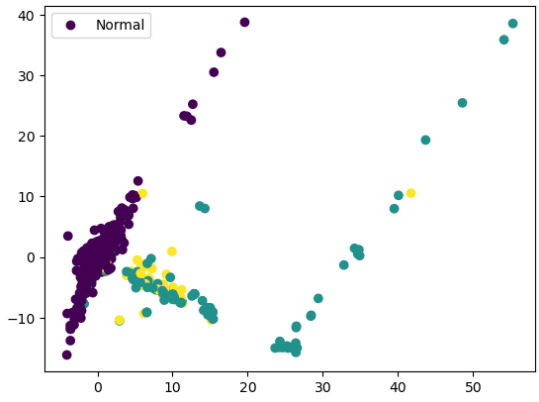


Fig. 36. Scatter plot of normal and outlier data points. Violete are true negatives, yellow are false negatives and green are true positives.

Figure 37 presents a Venn diagram that illustrates the intersection of detected outliers by an Autoencoder, Isolation Forest, and Neural Network (NN). This diagram highlights the consensus on certain points classified as outliers, as well as the unique detections by each model.

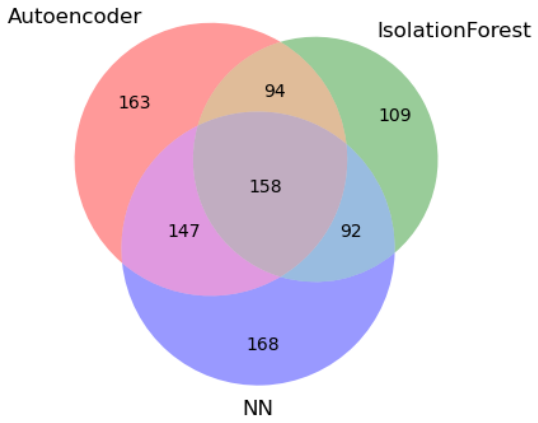


Fig. 37. Venn diagram showing the overlap of outlier detection between models.

The scatter plot indicates that while there is a cluster of commonly identified outliers, several points are uniquely identified by only one of the models. The Venn diagram further quantifies this overlap and distinction, suggesting that an ensemble approach could leverage the strengths of each individual model. For instance, points identified by all three models could be considered with high confidence as outliers, whereas points detected by only one model might require additional scrutiny.

Ensemble Model	ROC AUC	F1 Score	Fraud Detected
Ensemble 1	0.9095	0.1225	173 of 210
Ensemble 2	0.9417	0.271	171 of 210

TABLE VI
COMPARISON OF ENSEMBLE MODELS USING DATA BALANCED WITH 30% SMOTE.

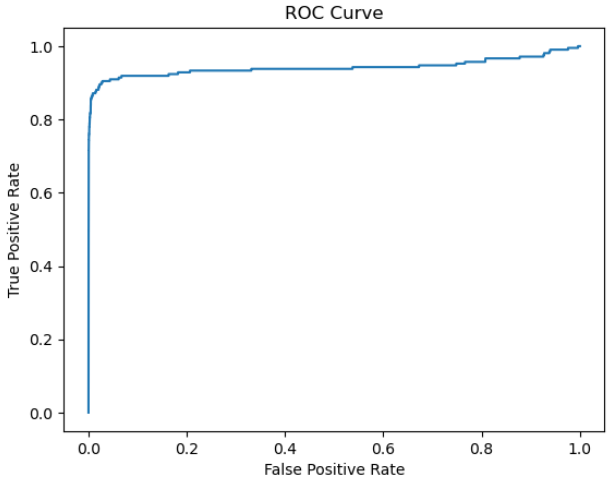


Fig. 38. ROC Curve of the Ensemble 2 model, demonstrating its performance.

Analysis of Ensemble Model Performance: The application of ensemble methods on a dataset balanced with 30% SMOTE yielded intriguing results. In the first ensemble, a weighted approach was taken, giving equal importance to the Isolation Forest and Neural Network models (each weighted at 0.25), and a higher weight to the SVM model (0.5). This ensemble detected 173 out of 210 outliers, achieving an F1 score of 0.1225. The relatively low F1 score suggests that while the ensemble was adept at identifying outliers, it struggled to maintain a balance between precision and recall.

The second ensemble strategy adjusted the weights slightly by incorporating predictions from an Autoencoder model, contributing a weight of 0.10, while slightly reducing the contributions from the Isolation Forest and Neural Network to 0.20 each, and maintaining the SVM weight at 0.5. This ensemble detected 171 out of 210 outliers but achieved a higher F1 score of 0.271. The inclusion of the Autoencoder seems to have enhanced the ensemble's performance in terms of F1 score, which indicates a better balance between precision and recall.

Note that the Isolation Forest result was the normalization between 0 and 1 of the scores of the decision function. The decision function indicates the distance of a sample from the decision boundary.

These outcomes suggest that while both ensembles performed similarly in detecting the correct number of outliers, the second ensemble was more successful in achieving a harmonious balance between the frequency and probability of accurate outlier detection. Also a better ROC AUC score. This highlights the impact that subtle variations in model weighting can have on ensemble performance and underscores the potential benefit of incorporating diverse models to capture a broad spectrum of data patterns.

The use of an Feed Forward Network can be used in a latter work to improve the balance of ensemble models.

VI. Conclusion

In conclusion, this research has offered valuable insights into the effectiveness of different machine learning models for fraud detection. By employing the SMOTE algorithm to balance the dataset, notable enhancements in predictive performance were observed across several models. Notably, the Support Vector Classifier (SVC) emerged as a standout performer, showcasing its superior ability to identify fraudulent transactions. The challenges encountered with models such as OneClassSVM and Isolation Forest underscore the complexities inherent in fraud detection tasks, emphasizing the importance of selecting models tailored to specific dataset characteristics.

VII. Future Work

One area yet to be fully explored is the realm of ensemble methods. These techniques, which amalgamate the predictive capabilities of diverse algorithms, hold the promise of boosting both the accuracy and resilience of fraud detection systems. Future research will delve into the construction of sophisticated ensemble frameworks, aiming to capitalize on this untapped potential.

Moreover, there is an opportunity to refine the approach to class imbalance—a persistent challenge in fraud detection—through advanced oversampling techniques like SMOTE or ADASYN, coupled with intricate undersampling strategies. This dual approach could lead to significant enhancements in model precision and recall.

Another avenue for expansion lies in the application of more unsupervised learning methods. Techniques such as anomaly detection algorithms like One-Class Random Forest and more fine-tuning of the models I used could provide fresh perspectives on detecting patterns of fraudulent behavior without the need for labeled data.

The initial ensemble results did not meet the high expectations set for this project. Recognizing this as a preliminary attempt, future endeavors will be directed towards refining these methods, with the confidence that iterative improvements will lead to a robust solution capable of addressing the complexities of fraud detection.

References

[1] R. Teuwens, "Anomaly Detection with Auto-Encoders," Kaggle, 2023. [Online]. Available: https://www.kaggle.com/code/robinteuwens/anomaly-detection-with-auto-encoders.

[2] J. Brownlee, "SMOTE for Imbalanced Classification with Python," Machine Learning Mastery, 2023. [Online]. Available: https://machinelearningmastery.com/smote-oversampling-for-imbalanced-classification/.

[3] "Support Vector Machines Tutorial," SVM Tutorial, 2023. [Online]. Available: https://www.svm-tutorial.com/.

[4] A. Shafi, "Random Forest Classification with Scikit-Learn," DataCamp, Feb. 2023. [Online]. Available: https://www.datacamp.com/tutorial/random-forests-classifier-python.

[5] J. Brownlee, "Undersampling Algorithms for Imbalanced Classification," Machine Learning Mastery, 2023. [Online]. Available: https://machinelearningmastery.com/undersampling-algorithms-for-imbalanced-classification/.

Appendix

Annex A

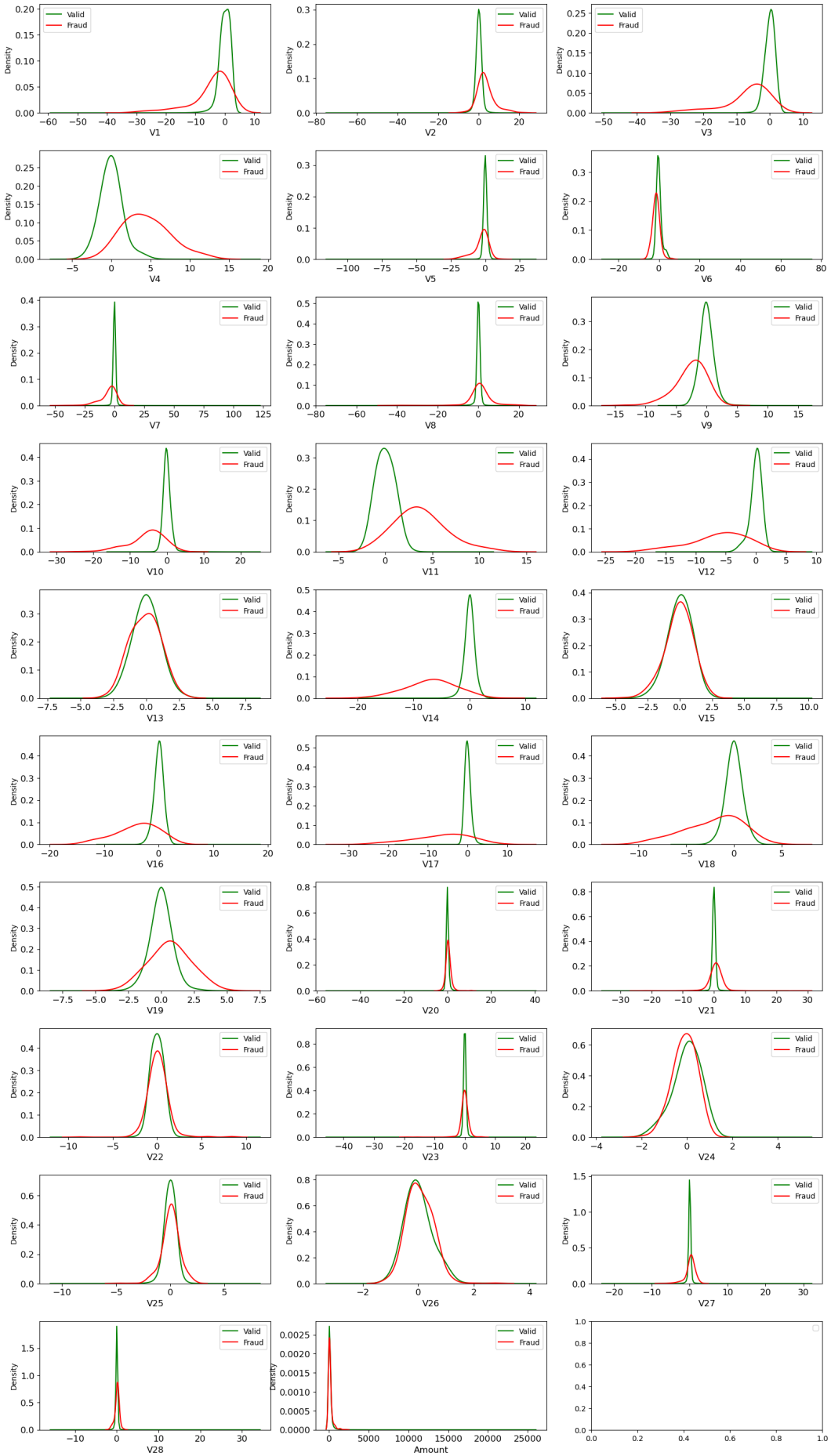


Fig. 39. Distributions of all features for fraud and valid transactions.

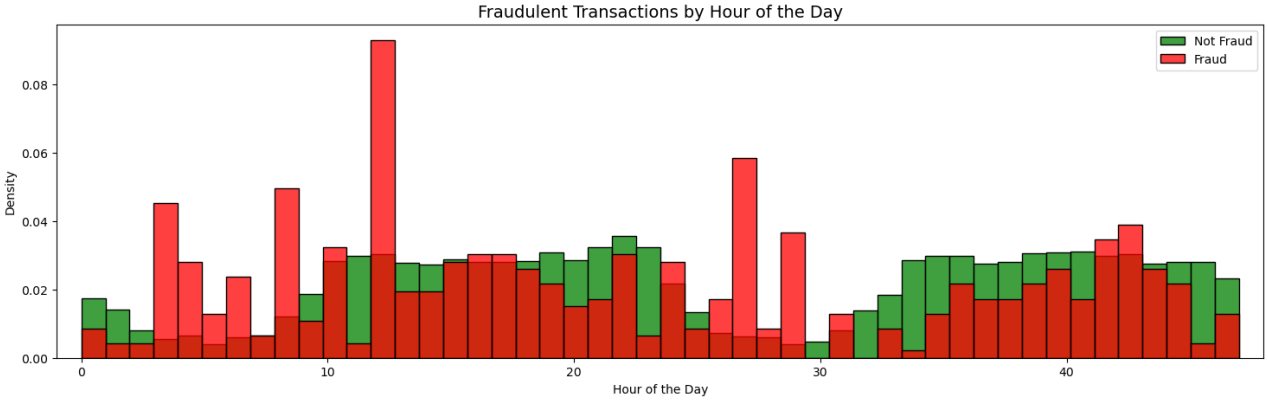


Fig. 40. 48 Hourly density of valid and fraudulent transactions highlighting the distinct patterns of activity.

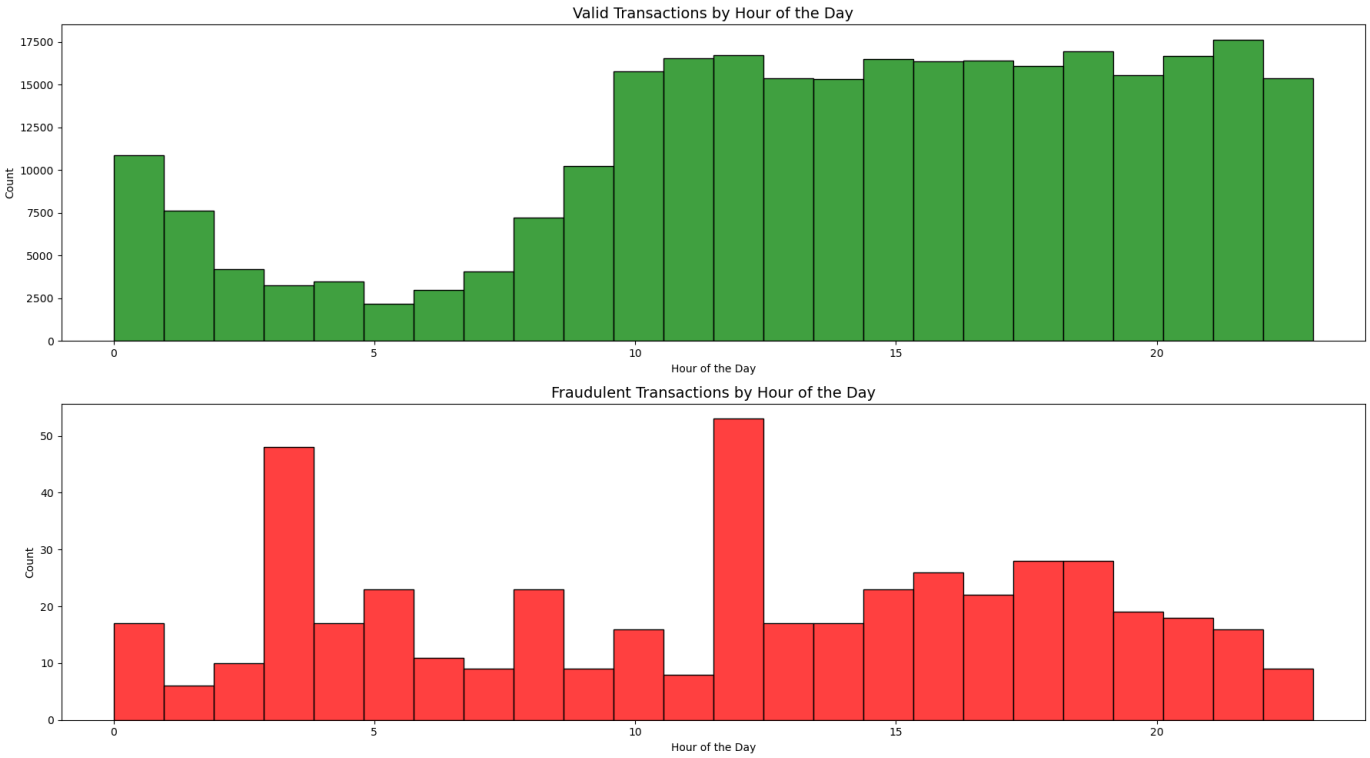


Fig. 41. 48 Hourly frequency of valid and fraudulent transactions highlighting the distinct patterns of activity.

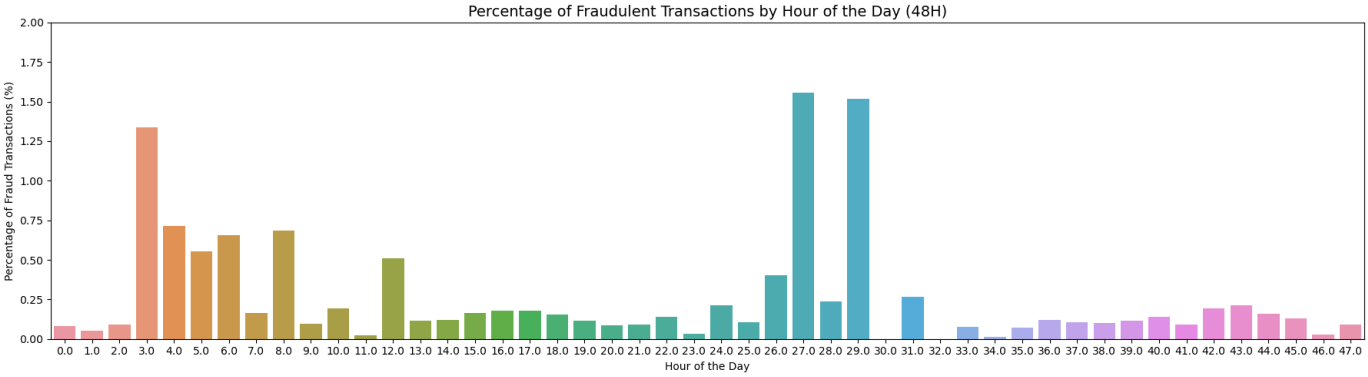


Fig. 42. Fraud by hour.

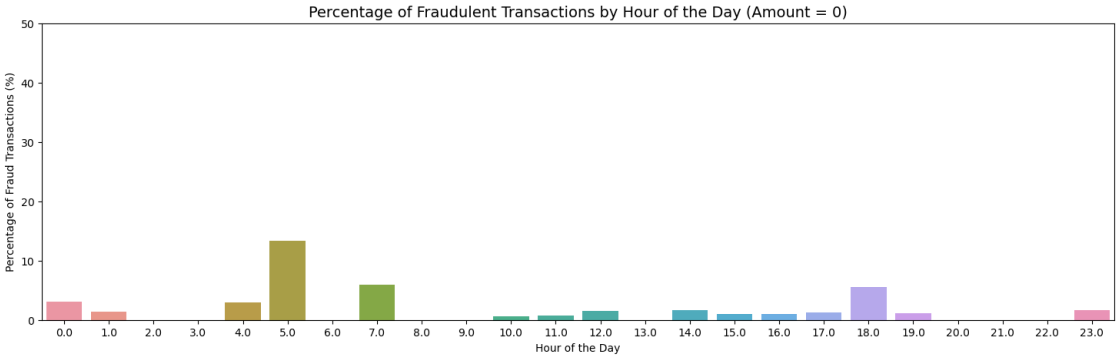


Fig. 43. Fraud by hour when the amount is 0